

LUT core-module ablation: three generations of lookup math

Comparison table, version 2

2026-09-11

Protocol (fixed for the whole paper set)

$n = 8$ **anchor pairs per table (nap, $K = 2^n = 256$ cells), 16,000 steps, batch 48 rows $\times$ 512 tokens, corrected eval** (48 rows $\times$ 100 steps with the leading 12 rows skipped), target hardware nebius-h100. **Every validation-bpb cell below is TBD.** Earlier numbers are not carried over, because none was measured on this protocol: the two prior Gen-2 results (2.1: 1.2281, `exp_g_0014`; 2.2: 1.2176, `exp_n_0044`) ran at $n = 6$ with the old batch-coupled eval and tied embeddings, and the Gen-3 runs `exp_g_0207/exp_g_0206` are 48K-step runs.

Setup and notation

Shared model. LUT transformer, $E = 384$, 6 layers, 6 attention heads. The FFN is `CompressionMultiHeadLUT` with $H = 8$ heads, $tph = 64$ tables per head, $d_{in} = d_{out} = 48$: `compress` $E \rightarrow Hd_{in}$, the *core LUT* on $[T, H, d_{in}] \rightarrow [T, H, d_{out}]$, `decompress` $Hd_{out} \rightarrow E$. Only the core LUT differs between rows. Per head h , the core reads $z_h \in \mathbb{R}^{d_{in}}$.

Per table t (of the tph tables of head h): n fixed anchor pairs $(a_j, b_j) \subset [0, d_{in})$, $u_j = z_h[a_j] - z_h[b_j]$, $m_j = |u_j|$, bits $\beta_j = [u_j > 0]$, cell $c = \text{pack}(\beta) \in [0, K)$, table $W_t \in \mathbb{R}^{K \times d_{out}}$. $c' = c^{(j^*)}$ is c with the bit of the least-confident pair $j^* = \arg \min_j |u_j|$ flipped. The head output is $y_h = \sum_{t \in h} (\cdot)$. In the backward, $g = \partial L / \partial y_h$, $G_t(\ell) = \langle g, W_t[\ell] \rangle$, and any $\partial L / \partial u_j$ is scattered as $+$ to $z_h[a_j]$ and $-$ to $z_h[b_j]$. σ is the logistic sigmoid. The cell index is integer-valued: no gradient ever flows through it (no straight-through estimator in any row). $P = H \cdot tph = 512$ tables per token and $d = d_{out}$.

Counting convention for FLOPs and MACs

- **Scope:** the core LUT module only, **per token**, one layer. Multiply by T for a sequence and by 6 for the model. The `compress/decompress` projections are identical in every row and excluded: they add $E \cdot Hd_{in} + Hd_{out} \cdot E = 294,912$ MACs ($\approx 589,824$ FLOPs) forward and about twice that backward per token per layer, which is *more* than the forward of every core variant below.
- **FLOP** = one scalar floating-point operation: $+$, $-$, $\times$, $\div$, a float comparison, $|\cdot|$, $\exp$, $\log \sigma$, σ each count 1. **MAC** = a multiply whose result is accumulated (weight $\times$ value, then added): **1 MAC = 2 FLOPs**, and MACs are included in the FLOP column. A pure accumulate (adding a looked-up row) is 1 FLOP and not a MAC.
- **Not counted:** memory reads (table-row gathers, anchor gathers), integer work (bit packing, XOR, index offsets), optimizer updates, and saving activations.
- **Forward** = inference cost (what an evaluated model executes). **Backward** = all *additional* arithmetic a training step needs, including training-only forward work (for example the alternative search in 1.1).
- Terms proportional to d , K and nK are exact. **Scalar per-table overheads** ($O(n)$ and constants) are counted from the code but approximate to within a few operations per table, since autograd’s internal breakdown varies. d_{in} does not enter the core cost: it only sets the anchor index range.

- In the Gen-2 and Gen-3 cost expressions, k is the number of table cells whose rows enter the computation (for example $k = 2$ for a two-cell blend). Gen-1 expressions are written with their fixed two alternatives substituted. Temperature gradients ($O(1)$ per layer) are ignored. Gen-2 rows assume the confidence gate is *off*, as in the Gen-2 runs.

Gen-3 confidence forms

LightMHL multiplies each addressed row by a score computed from the margins. The score is computed at its call site `light_multi_head_lut.py:549-550`; it is consumed by `_bagged_sum` at :565 for $n = 1$, and by `_blend_bag` at :557-561 (as `score · w`, :677) for $n = 2$. The function is `_confidence_score`, imported at `light_multi_head_lut.py:38` and defined at `fast_multi_head_lut.py:171-185`¹. Three forms exist ($g = \text{confidence_gain}$, default 1):

form	score	score at a boundary / before ²	configs using it (of 55 Gen-3)
<code>bounded</code>	$g \prod_j \sigma(2m_j)$	0.63	0
<code>bounded_norm</code>	$g \left(\prod_j \sigma(2m_j) \right)^{1/n}$ (geometric mean)	0.94	8
<code>margin</code>	$g \left(\sum_j m_j \right) \prod_j \sigma(2m_j)$ (LookupFFN kernel)	0.56	47 (including all four runs cited here ³)
<code>min_margin</code> (<i>proposed</i> , row 3.3)	$g \left(\min_j m_j \right) \prod_j \sigma(2m_j)$	0	not implemented

None of the three existing forms vanishes at a cell boundary, so all of them leave the $n = 1$ forward discontinuous; only `min_margin` reaches 0 there. For $n = 1$ inference on CUDA without grad, the score is computed by a separate native kernel⁴.

¹`_confidence_score` is a shared helper used by FastMHL, LightMHL and BH4 (`bh4_multi_head_lut.py:56`, 213). It lives in the `fast` module for historical reasons and is arguably misplaced there; a common module would be the natural home.

²Median ratio of the score with one margin set to 0 to the score before, on random nap-8 margins.

³`exp_g_0193`, `exp_g_0194`, `exp_g_0206`, `exp_g_0207`.

⁴`native/lutorch/lutorch.cu:241-247`, reached via `light_multi_head_lut.py:700-707` (`_fused_eval`). $n = 1$ eval and $n = 1$ training therefore compute the score through different code, which agrees to $\approx 2e-7$ in fp32. The bpb numbers come from the eval path. Gen 2 never uses this scored kernel.

Table A — forward and backward math

Variant	Forward pass (math)	Backward pass (math)
GEN 1 — Spiking Manifesto basic model, rational uncertainty $U(u) = 0.5/(1 + u)^a$ — both rows use exactly two alternatives: c and c'		
1.1 Hard forward, two-alternatives backward	$y_h = \sum_t W_t[c_t]$. Training only: the alternative is selected by $j^* = \arg \min_j u_j $.	Weights: $\partial L / \partial W_t[c_t] += g$ (addressed row only). Input (surrogate — the derivative of 1.2’s blend): $\frac{\partial L}{\partial u_{j^*}} = -U'(u_{j^*}) (G_t(c_t) - G_t(c'_t))$, with $-U'(u) = \frac{0.5 \operatorname{sgn} u}{(1 + u)^2}$; $\partial L / \partial u_j = 0$ for $j \neq j^*$.
1.2 Soft forward, two-alternatives backward	$y_h = \sum_t [(1 - U_t) W_t[c_t] + U_t W_t[c'_t]]$, $U_t = U(u_{t,j^*})$. Continuous at $u_{j^*} = 0$ (both weights $\frac{1}{2}$). Applied at eval too.	Weights: $\partial L / \partial W_t[c_t] += (1 - U_t) g$, $\partial L / \partial W_t[c'_t] += U_t g$. Input: $\partial L / \partial u_{j^*} = -U'(u_{j^*}) (G_t(c_t) - G_t(c'_t))$ as in 1.1, which here is the <i>exact</i> gradient of the blend with both cells held fixed.
GEN 2 — FastMHL math^b . Surrogate: $\rho_j = \frac{ u_j }{T_s + u_j }$, $s_\ell = \sum_j \rho_j - 2 \sum_{j: \operatorname{bit}_j(\ell) \neq \beta_j} \rho_j$, $\pi_\ell = \operatorname{softmax}_\ell(s_\ell / T_{sel})$ (signs β pinned; T_s, T_{sel} are temperatures)		
2.1 Hard forward, all-alternatives backward (all K cells)	$y_h = \sum_t W_t[c_t]$. ($c_t = \arg \max_\ell s_\ell$; no K -sized tensor is built.)	Weights: $\partial L / \partial W_t[c_t] += g$. Input via the K -cell surrogate $\hat{y}_t = \sum_\ell \pi_\ell W_t[\ell]$ (never used in the forward): $\frac{\partial L}{\partial u_j} = \frac{1}{T_{sel}} \sum_\ell \pi_\ell (G_t(\ell) - \bar{G}_t) \frac{\partial s_\ell}{\partial \rho_j} \cdot \frac{T_s \operatorname{sgn} u_j}{(T_s + u_j)^2}$, $\bar{G}_t = \sum_\ell \pi_\ell G_t(\ell)$, $\partial s_\ell / \partial \rho_j = \pm 1$ (agree/disagree with β_j). All n pairs get gradient; reads all K rows.
2.2 Hybrid_smooth two-alternatives forward, all-alternatives backward	$y_h = \sum_t [(1 - w_t) W_t[c_t] + w_t W_t[c'_t]]$, $w_t = \sigma(-\Delta_t / T_{sel})$, $\Delta_t = 2\rho_{t,j^*}$ (the exact 2-way softmax of s over $\{c, c'\}$). Applied at eval too.	Weights: $(1 - w_t) g$ into $W_t[c_t]$ and $w_t g$ into $W_t[c'_t]$. Input: the K -cell surrogate of 2.1 (not the gradient of the forward that actually ran).
2.3 Hard forward, 2-alternatives backward	$y_h = \sum_t W_t[c_t]$	TBD — not implemented; open decision. FastMultiHeadLut raises ValueError for backward_topk>0 unless forward_mode='hybrid_smooth' (fast_multi_head_lut.py:1305).
2.4 Hybrid_smooth two-alternatives forward, 2-alternatives backward	As 2.2.	Weights: as 2.2. Input (backward_topk=1): the exact gradient of the 2-cell blend with cells fixed. Only j^* gets gradient: $\frac{\partial L}{\partial u_{j^*}} = -\frac{w_t(1 - w_t)}{T_{sel}} (G_t(c'_t) - G_t(c_t)) \frac{2T_s \operatorname{sgn} u_{j^*}}{(T_s + u_{j^*})^2}$.
GEN 3 — LightMHL math (LookupFFN-inspired) . Score s_t per the confidence-form table above; the cell index uses detached u^c		
3.1 Soft forward $n=1$, margin confidence, single-alternative backward	$y_h = \sum_t s_t W_t[c_t]$, $s_t = (\sum_j u_{t,j}) \prod_j \sigma(2 u_{t,j})$. Piecewise-continuous: the margin score stays positive at a sign flip, so y jumps by $s_t (W_t[c'] - W_t[c])$; a min-margin score would make it continuous (row 3.3) ^d .	Weights: $\partial L / \partial W_t[c_t] += s_t g$. Input (only through the score): $\frac{\partial L}{\partial u_j} = G_t(c_t) \left(\prod_i \sigma(2 u_i) + 2s_t \sigma(-2 u_j) \right) \operatorname{sgn} u_j$ for all j . $\partial L / \partial W_t[c']$ is exactly zero (verified by autograd), and perturbing $W_t[c']$ leaves the input gradient unchanged: neither forward nor backward carries any signal about the neighbouring cell. At $n = 2$ both do.
3.2 Soft forward $n=2$, margin confidence, two-alternatives backward	$y_h = \sum_t s_t [\omega_0 W_t[c_t] + \omega_1 W_t[c'_t]]$, $\omega_1 = \sigma(-2 u_{j^*} /\tau)$, $\omega_0 = 1 - \omega_1$, $\tau = e^{\log \tau}$ (learnable). Continuous at bit flips (the two cells swap roles at weights $\frac{1}{2}$), but it retains a smaller discontinuity where the two smallest margins swap and the second cell changes: about $8\times$ smaller than 3.1’s jump on trained weights ^d .	Weights: $s_t \omega_0 g$ and $s_t \omega_1 g$ into the two read cells. Input: $\frac{\partial L}{\partial u_j} = (\omega_0 G_0 + \omega_1 G_1) \frac{\partial s_t}{\partial u_j } \operatorname{sgn} u_j + [j = j^*] s_t \frac{2}{\tau} \omega_0 \omega_1 (G_0 - G_1) \operatorname{sgn} u_j$, with $G_0 = G_t(c_t)$, $G_1 = G_t(c'_t)$. $\partial L / \partial \log \tau = \sum_t s_t \omega_0 \omega_1 (G_1 - G_0) 2 u_{j^*} /\tau$.

continued on next page

Variant	Forward pass (math)	Backward pass (math)
3.3 Soft forward $n=1$, min-margin confidence, single-alternative backward (<i>proposed</i>)	$y_h = \sum_t \tilde{s}_t W_t[c_t]$, $\tilde{s}_t = g(\min_j u_{t,j}) P_t$, $P_t = \prod_j \sigma(2 u_{t,j})$, $g \approx 38$ (scale-matched to margin ^e). At a boundary ($ u_i \rightarrow 0$, i becomes the argmin): $\tilde{s}_t = \frac{g}{2} C_i u_i + O(u_i^2)$, $C_i = \prod_{j \neq i} \sigma(2 u_j)$, i.e. it vanishes linearly. So y is continuous everywhere (C^0 , Lipschitz, piecewise smooth) but not differentiable at a boundary: the one-sided slopes are $-\frac{g}{2} C_i W_t[c]$ and $+\frac{g}{2} C_i W_t[c']$. It also has kinks where the argmin switches.	Weights: $\partial L / \partial W_t[c_t] = \pm \tilde{s}_t g$. Input (only through the score): $\frac{\partial L}{\partial u_j} = G_t(c_t) (g P_t [j = j^*] + 2 \tilde{s}_t \sigma(-2 u_j)) \operatorname{sgn} u_j$, i.e. the gP term goes to the argmin anchor only. Ties use the first-index convention of <code>torch.min</code> (not <code>amin</code> , whose even split breaks the parity between Fast’s analytic backward and Light’s autograd). TBD — not yet implemented. Row 3.4 ($n = 2 + \text{min-margin}$) is deferred : $n = 2$ is already continuous at bit flips and min-margin does not remove its argmin-switch jump, so it would not test continuity. Run it only if 3.3 beats 3.1.

Notes to Table A.

- ^a Other uncertainty functions may be tried later. The code also offers `UncertaintyMode.INVERSE_QUADRATIC`, $0.5/(1+u^2)$; see also the prior-evidence appendix on confidence forms.
- ^b **Config trap (Gen 2 only):** `backward_topk` counts cells *beyond* the addressed one. Our “2 alternatives” is `backward_topk=1`; `backward_topk=2` would mean 3 cells. “All alternatives” is all $K = 2^n$ cells (`backward_topk=0`, a full softmax surrogate), not the n single-bit flips (that would be `backward_topk=n`). `backward_topk` never changes the weight gradient.
- ^c A hard-forward Gen-3 variant still needs to be designed: nothing in the code trains LightMHL with a hard forward.
- ^d **Measured** (`continuity_probe.py`, `continuity_probe_trained.py`). **3.1, paper config, random init, float64:** at the boundary the score is a median $0.50\times$ the typical table score, never 0. The jump is a median 5.4% of $\|y_h\|$ (p90 14%), against $4.6\text{e-}10$ for a same-size step with no flip, i.e. 8 orders of magnitude above the float floor. **3.1, trained (exp_g_0193,** real val tokens): median 3.5% of the head output and 0.9% of the FFN output after `decompress`; about 1.2% of real margins lie within 10^{-2} of zero. **3.2, trained (exp_g_0194):** the argmin-switch jump is a median 0.44% of the head output and 0.11% of the FFN output, about $8\times$ smaller.
- ^e The gain matches `margin`’s mean on the same margins; the ratio is stable through training (38.9 at init, 36.3 after 4K steps, 37.4 on `exp_g_0193`). **Confound:** `min_margin` changes continuity *and* selectivity together (within-token CV 1.7–2.0 vs 0.87–0.98 for `margin`; 10–27% of (token, table) scores below 10^{-3}), so a single 3.3-vs-3.1 arm cannot separate the two effects. Details: `notes_min_margin.md`.

Table B — cost and results

Per token, one layer, core LUT only. Concrete values at $H = 8$, $tpb = 64$, $d_{in} = d_{out} = 48$, $n = 8$ ($K = 256$), $P = 512$, $d = d_{out}$. Validation bpb on the protocol above.

Variant	Forward FLOPs	Backward FLOPs	Forward MACs	Backward MACs	Validation bpb	Implementation (class used)
1.1	$P(2n + d)$ = 32,768	$P(5d + 2n + 9)$ = 135,680	0	$P \cdot 2d$ = 49,152	TBD — not yet implemented in the FFN harness.	<code>spiky.lutorch.multi_head_lut.MultiHeadLut (smooth_mode=False, n_alternatives=1, uncertainty_mode=INVERSE_L1); also lut_fused.LUTLayer.LUTLayerBasic. Not wired into CompressionMultiHeadLUT.</code>
1.2	$P(4d + 4n + 4)$ = 116,736	$P(8d + 9)$ = 201,216	$P \cdot 2d$ = 49,152	$P \cdot 4d$ = 98,304	TBD — not yet implemented in the FFN harness.	<code>MultiHeadLut (smooth_mode=True, n_alternatives=1). Not wired into CompressionMultiHeadLUT.</code>
2.1	$P(2n + d)$ = 32,768	$P(2Kd + 4nK + 9K + 11n + d)$ = 18,026,496^f	0	$P(Kd + 2nK)$ = 8,388,608	TBD — not yet run	<code>CompressionMultiHeadLUT(lut_impl='fast') → FastMultiHeadLut(forward_mode='hard', backward_topk=0); keys lut_forward_mode, lut_backward_topk.</code>
2.2	$P(2kd + 4n + 5)$ = 117,248	$P(2Kd + 4nK + 9K + 11n + 2kd)$ = 18,100,224^f	Pkd = 49,152	$P(Kd + 2nK + kd)$ = 8,437,760	TBD — not yet run	<code>FastMultiHeadLut(forward_mode='hybrid_smooth', backward_topk=0).</code>
2.3	$P(2n + d) = 32,768$	n/a	0	n/a	TBD — not implemented; open decision	Would be <code>FastMultiHeadLut(forward_mode='hard', backward_topk=1)</code> , which the code rejects.
2.4	$P(2kd + 4n + 5)$ = 117,248	$P(4kd + 9n + 15)$ = 241,152	Pkd = 49,152	$P2kd$ = 98,304	TBD — not yet run	<code>FastMultiHeadLut(forward_mode='hybrid_smooth', backward_topk=1) (note b).</code>
3.1	$P(2kd + 7n)$ = 77,824	$P(4kd + 8n)$ = 131,072	Pkd = 24,576	$P2kd$ = 49,152	TBD — not yet run	<code>CompressionMultiHeadLUT(lut_impl='light') → LightMultiHeadLUT(confidence_form='margin', read_top_n=1).</code>
3.2	$P(2kd + 8n + 7)$ = 134,656	$P(4kd + 8n + 11)$ = 235,008	Pkd = 49,152	$P2kd$ = 98,304	TBD — not yet run	<code>LightMultiHeadLUT(confidence_form='margin', read_top_n=2, read_tau=0.5, learnable).</code>
3.3	as 3.1 = 77,824^g	as 3.1 = 131,072^g	24,576	49,152	TBD — not yet implemented; open decision	Would be <code>LightMultiHeadLUT(confidence_form='min_margin', confidence_gain≈38, read_top_n=1); touchpoints in notes_min_margin.md.</code>

Notes to Table B.

^f Dominated by $2Kd$: the all-alternatives backward reads all K rows of every table. Memory: at $n = 8$ it holds several [tokens, 512, 256] fp32 buffers, about 12.9 GiB each at 24,576 tokens, so batch 48 needs micro-batching or gradient accumulation.

^g Listed as identical to 3.1. A minimum over n margins costs the same $n - 1$ operations as a sum over n . In the backward the gP term lands on one anchor instead of n , which saves $P(n - 1) = 3,584$ FLOPs, but that is smaller than the stated scalar-overhead tolerance, so it is not claimed as a difference.

Open decisions

1. **Row 2.3** (hard forward, 2-alternatives backward) is rejected by `FastMultiHeadLut` (`ValueError` unless `forward_mode='hybrid_smooth'`). Either patch the code to make it legal, or drop the row.

2. **Row 3.3** (min-margin): implement it now, or keep it planned but not run. The spec, scale analysis and touchpoints are in `notes_min_margin.md`.
3. Rows 1.1/1.2 cannot run at all until Gen 1 is wired into `CompressionMultiHeadLUT`: `MultiHeadLut` takes a shared `[B, input_dim]` input rather than the per-head `[T,H,d_in]` layout (for example via `anchor_candidates` restricting each head’s anchors).

Where the code disagrees with the original brief

1. **Gen 1 is not wired into the experiment harness.** `CompressionMultiHeadLUT` accepts only `lut_impl ∈ {fast, light, bh4}` (`compression_mhl.py:184`), and no `model_build.py` key reaches `MultiHeadLut`.
2. **In 1.1 the input gradient is a surrogate.** It is the derivative of 1.2’s soft blend, while the weights update only the addressed row.
3. **The paper’s status report writes** $u(\delta) = 0.5/(T + |\delta|)$ with a per-LUT temperature. The code’s $U(u) = 0.5/(1 + |u|)$ has none (`l_projection.py:96`, `lutorch.cu:797`).
4. **2.3 cannot be built** (see Open decisions).
5. **Gen-2 alternative counting:** see note b. The off-by-one in `backward_topk` is a live trap when writing Gen-2 configs.
6. **Gen 2’s hybrid_smooth weights are** $w = \sigma(-2\rho_{j^*}/T_{sel})$, a temperature-controlled rational soft-sign, not Gen 1’s $U(u)$. It does match “two-alternatives forward”.
7. **3.1 “soft forward $n=1$ ” reads one hard cell scaled by a smooth score.** It is piecewise-continuous with measured jumps (note d), not continuous.
8. **The margin confidence function is LookupFFN’s kernel** $(\sum_j |u_j|) \prod_j \sigma(2|u_j|)$, not $\min_j |u_j|$. The factor 2 is fixed.
9. **Minor:** `exp_g_0207` (3.1) sets `lut_read_tau_learnable=true`, which creates a parameter that never receives a gradient at $n = 1$.

TBD cells and experiments still to run

- **Every validation-bpb cell is TBD** on the protocol ($n = 8, 16K$, batch 48, corrected eval).
- **Needs implementation first:** 1.1 and 1.2 (wiring); 2.3 and 3.3 (open decisions). Cost cells for 2.3 are n/a until it exists.
- **Needs a run:** 2.1, 2.2, 2.4, 3.1, 3.2.
- **Reference anchors** (vanilla dense FFN, corrected eval): 1.1651 / 1.1618 at 16K (two seeds), 1.1154 at 48K.

Appendix — prior evidence on confidence forms

Summarised from `experiments/ffn_replacement/runs_corrected/LOOKUPFFN_LINE.md` and `diag_confidence_forms.py`, which measure 6,291,456 real nap-8 margin vectors (`dump_margins.py`, model `sweep_s05_dout48_H4_tph256_c256_din32` at init). This partly answers note a’s “other uncertainty functions”. The regenerated cache reproduces the published numbers exactly (`min_margin_scale.py`).

Scale and within-token selectivity at init. The within-token spread is the part of the score that actually reweights the table ensemble; the across-token part only rescales a token’s whole FFN output.

form	mean	p75/p25	within-token CV	reading
<code>bounded</code>	0.0542	2.06	0.54	18.4× forward attenuation at $n = 8$
<code>margin</code>	0.2286	3.04	0.87	self-normalises in training (0.94 after 4K steps)
<code>bounded_norm</code>	0.6838	1.09	0.061	nearly constant per token, so the linear <code>decompress</code> absorbs it
<code>min_margin</code> (proposed)	0.0059	7.32	1.78	39× below <code>margin</code> ; needs $g \approx 38$ (note e)

4K-step confidence-gate arms vs the gate-off baseline S5 at **1.434572**: `bounded` unmatched +0.225 (stopped at 3,200 steps, widening); `bounded` × 12.61 +0.0004; `bounded_norm` +0.0066 (inside the 0.0096 seed sd); `bounded_norm` on the Light implementation +0.0431; `margin` × 2.99 −0.0021. **Correction to how these gains are usually quoted:** the × 12.61 and × 2.99 arms were scale-matched to `bounded_norm`’s mean **0.6838**, not to gate-off’s 1.0. So scale, not the form, broke `bounded`, and `margin` was the only gate to land below the baseline.

16K, LightMHL: the clean pair `exp_g_0189` (`bounded_norm`) → `exp_g_0193` (`margin`) differs in nothing but the form and moves bpb by **−0.0346** (10.3× the 0.00335 seed spread). This is why `margin` became the standard.